Lecture 6
Solutions

Homework 6: Sharing

Solutions to Homework 6

Part A.

```
$ bash codeHints/L06/setup_hw6.sh
$ cd practice
$ git clone hw6_server.git hw6
$ cd hw6
$ git remote -v                      # origin ... (fetch), origin ... (push)
$ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
$ echo "my work" > mine.txt
$ git add mine.txt
$ git commit -m "Add my work"
$ git status                         # "ahead of 'origin/main' by 1 commit"
$ printf 'remote=origin\nahead=1\nno-merge=fetch\n' > ../hw6_answers.txt
$ cd ../..                           # back to the course folder
$ bash codeHints/L06/friend.sh
$ cd practice/hw6
$ git push                           # rejected (fetch first)
$ git config pull.rebase false
$ git pull --no-edit                 # fetch + merge: one merge commit
$ git log --oneline --graph
$ git push
$ git switch -c feature
$ echo "feature" > feature.txt
$ git add feature.txt
$ git commit -m "Add feature file"
$ git push -u origin feature
```

6.1–6.2. `git clone` created `hw6` with the full history and a remote named `origin` that points at the server folder. The remote's name is `origin` (Git chooses it for a clone), hence `remote=origin`.

6.3. Your branch has one commit that `origin/main` lacks, so `git status` says "ahead by 1 commit" and `ahead=1`. Nothing is on the server yet.

6.4. The friend's commit reached the server after you cloned, so the two copies diverged and `git push` is rejected with "fetch first". Plain `git pull` on a recent Git stops with "Need to specify how to reconcile divergent branches" (after already fetching); `git config pull.rebase false` chooses merge. The merge commit is the *one* merge commit the checker expects. (`git fetch` followed by `git merge origin/main` gives the same result.)

6.5. After the merge your `main` is ahead of `origin/main`, and a push is now a fast-forward for the server, so it is accepted.

6.6. `-u` records that local `feature` tracks `origin/feature` (the checker reads `branch.feature.remote` and `branch.feature.merge` from your repository).

6.7. `fetch`: it downloads and moves `origin/main` but changes neither your branches nor your files. `git pull` also merges.

Part B.

```
$ cd practice
$ mkdir hw6_github
$ cd hw6_github
$ git init -b main
$ git config user.email "ID+YOURNAME@users.noreply.github.com"   # if you use the no-
    reply address
$ echo "# Hello GitHub" > README.md
$ git add README.md
$ git commit -m "Add README"
$ git remote add origin git@github.com:YOURNAME/hello-github.git
$ git push -u origin main
        (now edit README.md on github.com, "Commit changes")
$ git pull
$ git log --oneline                  # two commits
```

Replace `YOURNAME` by your GitHub user name. After the browser edit, GitHub is one commit ahead of you, so `git pull` brings it in as a fast-forward and your local `main` again equals GitHub's.

Common mistakes and fixes.

- Running `friend.sh` before cloning, or pushing before 6.4. Then there is no divergence and no merge commit, so the check "exactly one merge commit" fails. Run `setup_hw6.sh --reset` and follow the order.
- `Permission denied (publickey)` when pushing to GitHub: the key is not added to your account, or the remote uses the wrong address style. Test with `ssh -T git@github.com`; check `git remote -v`.
- Creating the GitHub repository *with* a README, then pushing: rejected, because GitHub has a commit you lack. Either create it empty (best) or run `git pull --no-rebase origin main --allow-unrelated-histories`; both sides have a `README.md`, so expect an add/add conflict there: resolve it as in Lecture 5, commit, and push.
- Using `git push --force` to "fix" a rejection: not allowed here, it erases others' work.